

程序员也都假设机器会使用这种右移。另一方面，对于无符号数，右移必须是逻辑的。

与 C 相比，Java 对于如何进行右移有明确的定义。表达是 $x \gg k$ 会将 x 算术右移 k 个位置，而 $x \ggg k$ 会对 x 做逻辑右移。

旁注 移动 k 位，这里 k 很大

对于一个由 w 位组成的数据类型，如果要移动 $k \geq w$ 位会得到什么结果呢？例如，计算下面的表达式会得到什么结果，假设数据类型 `int` 为 $w=32$ ：

```
int    lval = 0xFEDCBA98 << 32;
int    aval = 0xFEDCBA98 >> 36;
unsigned uval = 0xFEDCBA98u >> 40;
```

C 语言标准很小心地规避了说明在这种情况下该如何做。在许多机器上，当移动一个 w 位的值时，移位指令只考虑位移量的低 $\log_2 w$ 位，因此实际上位移量就是通过计算 $k \bmod w$ 得到的。例如，当 $w=32$ 时，上面三个移位运算分别是移动 0、4 和 8 位，得到结果：

```
lval    0xFEDCBA98
aval    0xFFEDCBA9
uval    0x00FEDCBA
```

不过这种行为对于 C 程序来说是没有保证的，所以应该保持位移量小于待移位值的位数。

另一方面，Java 特别要求位移数量应该按照我们前面所讲的求模的方法来计算。

旁注 与移位运算有关的操作符优先级问题

常常有人会写这样的表达式 $1 \ll 2 + 3 \ll 4$ ，本意是 $(1 \ll 2) + (3 \ll 4)$ 。但是在 C 语言中，前面的表达式等价于 $1 \ll (2 + 3) \ll 4$ ，这是由于加法(和减法)的优先级比移位运算要高。然后，按照从左至右结合性规则，括号应该是这样打的 $(1 \ll (2 + 3)) \ll 4$ ，得到的结果是 512，而不是期望的 52。

在 C 表达式中搞错优先级是一种常见的程序错误原因，而且常常很难检查出来。所以当拿不准的时候，请加上括号！

 **练习题 2.16** 填写下表，展示不同移位运算对单字节数的影响。思考移位运算的最好方式是使用二进制表示。将最初的值转换为二进制，执行移位运算，然后再转换回十六进制。每个答案都应该是 8 个二进制数字或者 2 个十六进制数字。

x		$x \ll 3$		$x \gg 2$ (逻辑的)		$x \gg 2$ (算术的)	
十六进制	二进制	二进制	十六进制	二进制	十六进制	二进制	十六进制
0xC3							
0x75							
0x87							
0x66							

2.2 整数表示

在本节中，我们描述用位来编码整数的两种不同的方式：一种只能表示非负数，而另一种能够表示负数、零和正数。后面我们将会看到它们在数学属性和机器级实现方面密切相关。我们还会研究扩展或者收缩一个已编码整数以适应不同长度表示的效果。

图 2-8 列出了我们引入的数学术语，用于精确定义和描述计算机如何编码和操作整数。这些术语将在描述的过程中介绍，图在此处列出作为参考。

符号	类型	含义
$B2T_w$	函数	二进制转补码
$B2U_w$	函数	二进制转无符号数
$U2B_w$	函数	无符号数转二进制
$U2T_w$	函数	无符号转补码
$T2B_w$	函数	补码转二进制
$T2U_w$	函数	补码转无符号数
$TMin_w$	常数	最小补码值
$TMax_w$	常数	最大补码值
$UMax_w$	常数	最大无符号数
$+^t_w$	操作	补码加法
$+^u_w$	操作	无符号数加法
$*^t_w$	操作	补码乘法
$*^u_w$	操作	无符号数乘法
$-^t_w$	操作	补码取反
$-^u_w$	操作	无符号数取反

图 2-8 整数的数据与算术操作术语。下标 w 表示数据表示中的位数

2.2.1 整型数据类型

C 语言支持多种整型数据类型——表示有限范围的整数。这些类型如图 2-9 和图 2-10 所示，其中还给出了“典型”32 位和 64 位机器的取值范围。每种类型都能用关键字来指定大小，这些关键字包括 `char`、`short`、`long`，同时还可以指示被表示的数字是非负数（声明为 `unsigned`），或者可能是负数（默认）。如图 2-3 所示，为这些不同的大小分配的字节数根据程序编译为 32 位还是 64 位而有所不同。根据字节分配，不同的大小所能表示的值的范围是不同的。这里给出来的唯一一个与机器相关的取值范围是大小指示符 `long` 的。大多数 64 位机器使用 8 个字节的表示，比 32 位机器上使用的 4 个字节的表示的取值范围大很多。

C数据类型	最小值	最大值
<code>[signed]char</code>	-128	127
<code>unsigned char</code>	0	255
<code>short</code>	-32 768	32 767
<code>unsigned short</code>	0	65 535
<code>int</code>	-2 147 483 648	2 147 483 647
<code>unsigned</code>	0	4 294 967 295
<code>long</code>	-2 147 483 648	2 147 483 647
<code>unsigned long</code>	0	4 294 967 295
<code>int32_t</code>	-2 147 483 648	2 147 483 647
<code>uint32_t</code>	0	4 294 967 295
<code>int64_t</code>	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
<code>uint64_t</code>	0	18 446 744 073 709 551 615

图 2-9 32 位程序上 C 语言整型数据类型的典型取值范围

C数据类型	最小值	最大值
[signed]char	-128	127
unsigned char	0	255
short	-32 768	32 767
unsigned short	0	65 535
int	-2 147 483 648	2 147 483 647
unsigned	0	4 294 967 295
long	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
unsigned long	0	18 446 744 073 709 551 615
int32_t	-2 147 483 648	2 147 483 647
uint32_t	0	4 294 967 295
int64_t	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
uint64_t	0	18 446 744 073 709 551 615

图 2-10 64 位程序上 C 语言整型数据类型的典型取值范围

图 2-9 和图 2-10 中一个很值得注意的特点是取值范围不是对称的——负数的范围比整数的范围大 1。当我们考虑如何表示负数的时候，会看到为什么会这样。

C 语言标准定义了每种数据类型必须能够表示的最小的取值范围。如图 2-11 所示，它们的取值范围与图 2-9 和图 2-10 所示的典型实现一样或者小一些。特别地，除了固定大小的数据类型是例外，我们看到它们只要求正数和负数的取值范围是对称的。此外，数据类型 `int` 可以用 2 个字节的数字来实现，而这几乎回退到了 16 位机器的时代。还可以看到，`long` 的大小可以用 4 个字节的数字来实现，对 32 位程序来说这是很典型的。固定大小的数据类型保证数值的范围与图 2-9 给出的典型数值一致，包括负数与正数的不对称性。

C数据类型	最小值	最大值
[signed]char	-127	127
unsigned char	0	255
short	-32 767	32 767
unsigned short	0	65 535
int	-32 767	32 767
unsigned	0	65 535
long	-2 147 483 647	2 147 483 647
unsigned long	0	4 294 967 295
int32_t	-2 147 483 648	2 147 483 647
uint32_t	0	4 294 967 295
int64_t	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
uint64_t	0	18 446 744 073 709 551 615

图 2-11 C 语言的整型数据类型的保证的取值范围。C 语言标准要求这些数据类型必须至少具有这样的取值范围

给 C 语言初学者 C、C++ 和 Java 中的有符号和无符号数

C 和 C++ 都支持有符号（默认）和无符号数。Java 只支持有符号数。

2.2.2 无符号数的编码

假设有一个整数数据类型有 w 位。我们可以将位向量写成 $\vec{x}$ ，表示整个向量，或者写成 $[x_{w-1}, x_{w-2}, \dots, x_0]$ ，表示向量中的每一位。把 $\vec{x}$ 看做一个二进制表示的数，就获得

了 $\vec{x}$ 的无符号表示。在这个编码中，每个位 x_i 都取值为 0 或 1，后一种取值意味着数值 2^i 应为数字值的一部分。我们用一个函数 $B2U_w$ (Binary to Unsigned 的缩写，长度为 w) 来表示：

原理：无符号数编码的定义

对向量 $\vec{x} = [x_{w-1}, x_{w-2}, \dots, x_0]$ ：

$$B2U_w(\vec{x}) \doteq \sum_{i=0}^{w-1} x_i 2^i \quad (2.1)$$

在这个等式中，符号“ $\doteq$ ”表示左边被定义为等于右边。函数 $B2U_w$ 将一个长度为 w 的 0、1 串映射到非负整数。举一个示例，图 2-11 展示的是下面几种情况下 $B2U$ 给出的从位向量到整数的映射：

$$\begin{aligned} B2U_4([0001]) &= 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 0 + 0 + 1 = 1 \\ B2U_4([0101]) &= 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 4 + 0 + 1 = 5 \\ B2U_4([1011]) &= 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11 \\ B2U_4([1111]) &= 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 2 + 1 = 15 \end{aligned} \quad (2.2)$$

在图中，我们用长度为 2^i 的指向右侧箭头的条表示每个位的位置 i 。每个位向量对应的数值就等于所有值为 1 的位对应的条的长度之和。

让我们来考虑一下 w 位所能表示的值的范围。最小值是用位向量 $[00 \dots 0]$ 表示，也就是整数值 0，而最大值是用位向量 $[11 \dots 1]$ 表示，也就是整数值

$UMax_w \doteq \sum_{i=0}^{w-1} 2^i = 2^w - 1$ 。以 4 位情况为例， $UMax_4 = B2U_4([1111]) = 2^4 - 1 = 15$ 。因此，函数 $B2U_w$ 能够被定义为一个映射 $B2U_w: \{0, 1\}^w \rightarrow \{0, \dots, 2^w - 1\}$ 。



图 2-12 $w=4$ 的无符号数示例。当二进制表示中位 i 为 1，数值就会相应加上 2^i

无符号数的二进制表示有一个很重要的属性，也就是每个介于 $0 \sim 2^w - 1$ 之间的数都有唯一一个 w 位的值编码。例如，十进制值 11 作为无符号数，只有一个 4 位的表示，即 $[1011]$ 。我们用数学原理来重点讲述它，先表述原理再解释。

原理：无符号数编码的唯一性

函数 $B2U_w$ 是一个双射。

数学术语双射是指一个函数 f 有两面：它将数值 x 映射为数值 y ，即 $y = f(x)$ ，但它也可以反向操作，因为对每一个 y 而言，都有唯一一个数值 x 使得 $f(x) = y$ 。这可以用反函数 f^{-1} 来表示，在本例中，即 $x = f^{-1}(y)$ 。函数 $B2U_w$ 将每一个长度为 w 的位向量都映射为 $0 \sim 2^w - 1$ 之间的一个唯一值；反过来，我们称其为 $U2B_w$ (即“无符号数到二进制”)，在 $0 \sim 2^w - 1$ 之间的每一个整数都可以映射为一个唯一的长度为 w 的位模式。

2.2.3 补码编码

对于许多应用，我们还希望表示负数值。最常见的有符号数的计算机表示方式就是补码 (two's-complement) 形式。在这个定义中，将字的最高有效位解释为负权 (negative weight)。我们用函数 $B2T_w$ (Binary to Two's-complement 的缩写，长度为 w) 来表示：

原理：补码编码的定义

对向量 $\vec{x} = [x_{w-1}, x_{w-2}, \dots, x_0]$ ：

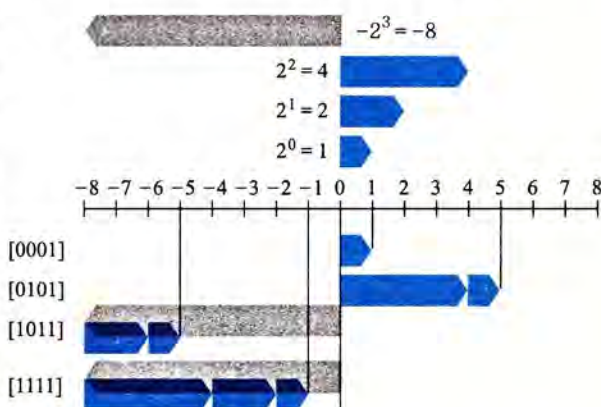
$$B2T_w(\vec{x}) \doteq -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \quad (2.3)$$

最高有效位 x_{w-1} 也称为符号位，它的“权重”为 -2^{w-1} ，是无符号表示中权重的负数。符号位被设置为 1 时，表示值为负，而当设置为 0 时，值为非负。这里来看一个示例，图 2-13 展示的是下面几种情况下 $B2T$ 给出的从位向量到整数的映射。

$$\begin{aligned} B2T_4([0001]) &= -0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 0 + 0 + 1 = 1 \\ B2T_4([0101]) &= -0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 4 + 0 + 1 = 5 \\ B2T_4([1011]) &= -1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 0 + 2 + 1 = -5 \\ B2T_4([1111]) &= -1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 4 + 2 + 1 = -1 \end{aligned} \quad (2.4)$$

在这个图中，我们用向左指的条表示符号位具有负权重。于是，与一个位向量相关联的数值是由可能的向左指的条和向右指的条加起来决定的。

我们可以看到，图 2-12 和图 2-13 中的位模式都是一样的，对等式 (2.2) 和等式 (2.4) 来说也是一样，但是当最高有效位是 1 时，数值是不同的，这是因为在一种情况中，最高有效位的权重是 +8，而在另一种情况中，它的权重是 -8。



让我们来考虑一下 w 位补码所能表示的值的范围。它能表示的最小值是位向量 $[10 \dots 0]$ (也就是设置这个位为负权，但是清除其他所有的位)，其整数值为 $TMin_w \doteq -2^{w-1}$ 。而最大值是位向量 $[01 \dots 1]$

(清除具有负权的位，而设置其他所有的位)，其整数值为 $TMax_w \doteq \sum_{i=0}^{w-2} 2^i = 2^{w-1} - 1$ 。以长度为 4 为例，我们有 $TMin_4 = B2T_4([1000]) = -2^3 = -8$ ，而 $TMax_4 = B2T_4([0111]) = 2^2 + 2^1 + 2^0 = 4 + 2 + 1 = 7$ 。

我们可以看出 $B2T_w$ 是一个从长度为 w 的位模式到 $TMin_w$ 和 $TMax_w$ 之间数字的映射，写作 $B2T_w: \{0, 1\}^w \rightarrow \{TMin_w, \dots, TMax_w\}$ 。同无符号表示一样，在可表示的取值范围内的每个数字都有一个唯一的 w 位的补码编码。这就导出了与无符号数相似的补码数原理：

原理：补码编码的唯一性

函数 $B2T_w$ 是一个双射。

我们定义函数 $T2B_w$ (即“补码到二进制”) 作为 $B2T_w$ 的反函数。也就是说，对于每个数 x ，满足 $TMin_w \leq x \leq TMax_w$ ，则 $T2B_w(x)$ 是 x 的(唯一的) w 位模式。

 **练习题 2.17** 假设 $w=4$ ，我们能给每个可能的十六进制数字赋予一个数值，假设用一个无符号或者补码表示。请根据这些表示，通过写出等式 (2.1) 和等式 (2.3) 所示的求和公式中的 2 的非零次幂，填写下表：

$\bar{x}$		$B2U_4(\bar{x})$	$B2T_4(\bar{x})$
十六进制	二进制		
0xE	[1110]	$2^3+2^2+2^1=14$	$-2^3+2^2+2^1=-2$
0x0			
0x5			
0x8			
0xD			
0xF			

图 2-14 展示了针对不同字长，几个重要数字的位模式和数值。前三个给出的是可表示的整数的范围，用 $UMax_w$ 、 $TMin_w$ 和 $TMax_w$ 来表示。在后面的讨论中，我们还会经常引用到这三个特殊的值。如果可以从上下文中推断出 w ，或者 w 不是讨论的主要内容时，我们会省略下标 w ，直接引用 $UMax$ 、 $TMin$ 和 $TMax$ 。

数	字长 w			
	8	16	32	64
$UMax_w$	0xFF 255	0xFFFF 65 535	0xFFFFFFFF 4 294 967 295	0xFFFFFFFFFFFFFFFF 18 446 744 073 709 551 615
$TMin_w$	0x80 -128	0x8000 -32 768	0x80000000 -2 147 483 648	0x8000000000000000 -9 223 372 036 854 775 808
$TMax_w$	0x7F 127	0x7FFF 32 767	0x7FFFFFFF 2 147 483 647	0x7FFFFFFFFFFFFFFF 9 223 372 036 854 775 807
-1	0xFF	0xFFFF	0xFFFFFFFF	0xFFFFFFFFFFFFFFFF
0	0x00	0x0000	0x00000000	0x0000000000000000

图 2-14 重要的数字。图中给出了数值和十六进制表示

关于这些数字，有几点值得注意。第一，从图 2-9 和图 2-10 可以看到，补码的范围是不对称的： $|TMin| = |TMax| + 1$ ，也就是说， $TMin$ 没有与之对应的正数。正如我们将会看到的，这导致了补码运算的某些特殊的属性，并且容易造成程序中细微的错误。之所以会有这样的不对称性，是因为一半的位模式（符号位设置为 1 的数）表示负数，而另一半（符号位设置为 0 的数）表示非负数。因为 0 是非负数，也就意味着能表示的整数比负数少一个。第二，最大的无符号数值刚好比补码的最大值的两倍大一点： $UMax_w = 2TMax_w + 1$ 。补码表示中所有表示负数的位模式在无符号表示中都变成了正数。图 2-14 也给出了常数 -1 和 0 的表示。注意 -1 和 $UMax$ 有同样的位表示——一个全 1 的串。数值 0 在两种表示方式中都是全 0 的串。

C 语言标准并没有要求要用补码形式来表示有符号整数，但是几乎所有的机器都是这么做的。程序员如果希望代码具有最大可移植性，能够在所有可能的机器上运行，那么除了图 2-11 所示的那些范围之外，我们不应该假设任何可表示的数值范围，也不应该假设符号数会使用何种特殊的表示方式。另一方面，许多程序的书写都假设用补码来表示有符号数，并且具有图 2-9 和图 2-10 所示的“典型的”取值范围，这些程序也能够大量的机器和编译器上移植。C 库中的文件 `<limits.h>` 定义了一组常量，来限定编译器运行的这台机器的不同整型数据类型的取值范围。比如，它定义了常量 `INT_MAX`、`INT_MIN` 和 `UINT_MAX`，它们描述了有符号和无符号整数的范围。对于一个补码的机器，数据类型 `int` 有 w 位，这些常量就对应于 $TMax_w$ 、 $TMin_w$ 和 $UMax_w$ 的值。

旁注 关于确定大小的整数类型的更多内容

对于某些程序来说,用某个确定大小的表示来编码数据类型非常重要。例如,当编写程序,使得机器能够按照一个标准协议在因特网上通信时,让数据类型与协议指定的数据类型兼容是非常重要的。我们前面看到了,某些C数据类型,特别是long型,在不同的机器上有不同的取值范围,而实际上C语言标准只指定了每种数据类型的最小范围,而不是确定的范围。虽然我们可以选择与大多数机器上的标准表示兼容的数据类型,但是这也不能保证可移植性。

我们已经见过了32位和64位版本的确定大小的整数类型(图2-3),它们是一个更大数据类型类的一部分。ISO C99标准在文件stdint.h中引入了这个整数类型类。这个文件定义了一组数据类型,它们的声明形如intN_t和uintN_t,对不同的N值指定N位有符号和无符号整数。N的具体值与实现相关,但是大多数编译器允许的值为8、16、32和64。因此,通过将它的类型声明为uint16_t,我们可以无歧义地声明一个16位无符号变量,而如果声明为int32_t,就是一个32位有符号变量。

这些数据类型对应着一组宏,定义了每个N的值对应的最小和最大值。这些宏名字形如INTN_MIN、INTN_MAX和UINTN_MAX。

确定宽度类型的带格式打印需要使用宏,以与系统相关的方式扩展为格式串。因此,举个例子来说,变量x和y的类型是int32_t和uint64_t,可以通过调用printf来打印它们的值,如下所示:

```
printf("x = %" PRIi32 ", y = %" PRIu64 "\n", x, y);
```

编译为64位程序时,宏PRIi32展开成字符串“d”,宏PRIu64则展开成两个字符串“l”“u”。当C预处理器遇到仅用空格(或其他空白字符)分隔的一个字符串常量序列时,就把它们串联起来。因此,上面的printf调用就变成了:

```
printf("x = %d, y = %lu\n", x, y);
```

使用宏能保证:不论代码是如何被编译的,都能生成正确的格式字符串。

关于整数数据类型的取值范围和表示,Java标准是非常明确的。它要求采用补码表示,取值范围与图2-10中64位的情况一样。在Java中,单字节数据类型称为byte,而不是char。这些非常具体的要求都是为了保证无论在什么机器上运行,Java程序都能表现地完全一样。

旁注 有符号数的其他表示方法

有符号数还有两种标准的表示方法:

反码(Ones' Complement):除了最高有效位的权是 $-(2^{w-1}-1)$ 而不是 -2^{w-1} ,它和补码是一样的:

$$B2O_w(\vec{x}) \doteq -x_{w-1}(2^{w-1}-1) + \sum_{i=0}^{w-2} x_i 2^i$$

原码(Sign-Magnitude):最高有效位是符号位,用来确定剩下的位应该取负权还是正权:

$$B2S_w(\vec{x}) \doteq (-1)^{x_{w-1}} \cdot \left(\sum_{i=0}^{w-2} x_i 2^i \right)$$

这两种表示方法都有一个奇怪的属性,那就是对于数字0有两种不同的编码方式。这两种表示方法,把[00...0]都解释为+0。而值-0在原码中表示为[10...0],在反码中表示为[11...1]。虽然过去生产过基于反码表示的机器,但是几乎所有的现代机器都使用补码。我们将看到在浮点数中有使用原码编码。

请注意补码(Two's complement)和反码(Ones' complement)中撇号的位置是不同的。术语补码来源于这样一个情况,对于非负数 x ,我们用 $2^w - x$ (这里只有一个 2)来计算 $-x$ 的 w 位表示。术语反码来源于这样一个属性,我们用 $[111\cdots 1] - x$ (这里有很多个 1)来计算 $-x$ 的反码表示。

为了更好地理解补码表示,考虑下面的代码:

```
1      short x = 12345;
2      short mx = -x;
3
4      show_bytes((byte_pointer) &x, sizeof(short));
5      show_bytes((byte_pointer) &mx, sizeof(short));
```

当在大端法机器上运行时,这段代码的输出为 30 39 和 cf c7,指明 x 的十六进制表示为 0x3039,而 mx 的十六进制表示为 0xcfc7。将它们展开为二进制,我们得到 x 的位模式为 $[0011000000111001]$,而 mx 的位模式为 $[1100111111000111]$ 。如图 2-15 所示,等式(2.3)对这两个位模式生成的值为 12 345 和 -12 345。

权	12 345		-12 345		53 191	
	位	值	位	值	位	值
1	1	1	1	1	1	1
2	0	0	1	2	1	2
4	0	0	1	4	1	4
8	1	8	0	0	0	0
16	1	16	0	0	0	0
32	1	32	0	0	0	0
64	0	0	1	64	1	64
128	0	0	1	128	1	128
256	0	0	1	256	1	256
512	0	0	1	512	1	512
1 024	0	0	1	1 024	1	1 024
2 048	0	0	1	2 048	1	2 048
4 096	1	4096	0	0	0	0
8 192	1	8192	0	0	0	0
16 384	0	0	1	16 384	1	16 384
±32 768	0	0	1	-32 768	1	32 768
总计	12 345		-12 345		53 191	

图 2-15 12 345 和 -12 345 的补码表示,以及 53 191 的无符号表示。注意后面两个数有相同的位表示

 **练习题 2.18** 在第 3 章中,我们将看到由反汇编器生成的列表,反汇编器是一种可将执行程序文件转换回可读性更好的 ASCII 码形式的程序。这些文件包含许多十六进制数字,都是用典型的补码形式来表示这些值。能够认识这些数字并理解它们的意义(例如它们是正数还是负数),是一项重要的技巧。

在下面的列表中,对于标号为 A~I(标记在右边)的那些行,将指令名(sub、mov 和 add)右边显示的(32 位补码形式表示的)十六进制值转换为等价的十进制值。

```
4004d0: 48 81 ec e0 02 00 00      sub    $0x2e0,%rsp          A.
4004d7: 48 8b 44 24 a8            mov    -0x58(%rsp),%rax     B.
4004dc: 48 03 47 28              add    0x28(%rdi),%rax      C.
4004e0: 48 89 44 24 d0            mov    %rax,-0x30(%rsp)     D.
```



```

4004e5: 48 8b 44 24 78      mov     0x78(%rsp),%rax      E.
4004ea: 48 89 87 88 00 00    mov     %rax,0x88(%rdi)      F.
4004f1: 48 8b 84 24 f8 01 00 mov     0x1f8(%rsp),%rax      G.
4004f8: 00
4004f9: 48 03 44 24 08      add     0x8(%rsp),%rax
4004fe: 48 89 84 24 c0 00 00 mov     %rax,0xc0(%rsp)      H.
400505: 00
400506: 48 8b 44 d4 b8      mov     -0x48(%rsp,%rdx,8),%rax I.

```

2.2.4 有符号数和无符号数之间的转换

C 语言允许在各种不同的数字数据类型之间做强制类型转换。例如，假设变量 x 声明为 `int`， u 声明为 `unsigned`。表达式 `(unsigned)x` 会将 x 的值转换成一个无符号数值，而 `(int)u` 将 u 的值转换成一个有符号整数。将有符号数强制类型转换成无符号数，或者反过来，会得到什么结果呢？从数学的角度来说，可以想象到几种不同的规则。很明显，对于在两种形式中都能表示的值，我们是想要保持不变的。另一方面，将负数转换成无符号数可能会得到 0。如果转换的无符号数太大以至于超出了补码能够表示的范围，可能会得到 $TMax$ 。不过，对于大多数 C 语言的实现来说，对这个问题的回答都是从位级角度来考虑的，而不是数的角度。

比如说，考虑下面的代码：

```

1      short   int    v = -12345;
2      unsigned short uv = (unsigned short) v;
3      printf("v = %d, uv = %u\n", v, uv);

```

在一台采用补码的机器上，上述代码会产生如下输出：

```
v = -12345, uv = 53191
```

我们看到，强制类型转换的结果保持位值不变，只是改变了解释这些位的方式。在图 2-15 中我们看到过， $-12\,345$ 的 16 位补码表示与 $53\,191$ 的 16 位无符号表示是完全一样的。将 `short` 强制类型转换为 `unsigned short` 改变数值，但是不改变位表示。

类似地，考虑下面的代码：

```

1      unsigned u = 4294967295u; /* UMax */
2      int      tu = (int) u;
3      printf("u = %u, tu = %d\n", u, tu);

```

在一台采用补码的机器上，上述代码会产生如下输出：

```
u = 4294967295, tu = -1
```

从图 2-14 我们可以看到，对于 32 位字长来说，无符号形式的 $4\,294\,967\,295 (UMax_{32})$ 和补码形式的 -1 的位模式是完全一样的。将 `unsigned` 强制类型转换成 `int`，底层的位表示保持不变。

对于大多数 C 语言的实现，处理同样字长的有符号数和无符号数之间相互转换的一般规则是：数值可能会改变，但是位模式不变。让我们用更数学化的形式来描述这个规则。我们定义函数 $U2B_w$ 和 $T2B_w$ ，它们将数值映射为无符号数和补码形式的位表示。也就是说，给定 $0 \leq x \leq UMax_w$ 范围内的一个整数 x ，函数 $U2B_w(x)$ 会给出 x 的唯一的 w 位无符号表示。相似地，当 x 满足 $TMin_w \leq x \leq TMax_w$ ，函数 $T2B_w(x)$ 会给出 x 的唯一的 w 位补码表示。

现在，将函数 $T2U_w$ 定义为 $T2U_w(x) \doteq B2U_w(T2B_w(x))$ 。这个函数的输入是一个

$TMin_w \sim TMax_w$ 的数, 结果得到一个 $0 \sim UMax_w$ 的值, 这里两个数有相同的位模式, 除了参数是无符号的, 而结果是以补码表示的。类似地, 对于 $0 \sim UMax_w$ 之间的值 x , 定义函数 $U2T_w$ 为 $U2T_w(x) \triangleq B2T_w(U2B_w(x))$ 。生成一个数的无符号表示和 x 的补码表示相同。

继续我们前面的例子, 从图 2-15 中, 我们看到 $T2U_{16}(-12\ 345) = 53\ 191$, 并且 $U2T_{16}(53\ 191) = -12\ 345$ 。也就是说, 十六进制表示写作 $0xCFC7$ 的 16 位位模式既是 $-12\ 345$ 的补码表示, 又是 $53\ 191$ 的无符号表示。同时请注意 $12\ 345 + 53\ 191 = 65\ 536 = 2^{16}$ 。这个属性可以推广到给定位模式的两个数值(补码和无符号数)之间的关系。类似地, 从图 2-14 我们看到 $T2U_{32}(-1) = 4\ 294\ 967\ 295$, 并且 $U2T_{32}(4\ 294\ 967\ 295) = -1$ 。也就是说, 无符号表示中的 $UMax$ 有着和补码表示的 -1 相同的位模式。我们在这两个数之间也能看到这种关系: $1 + UMax_w = 2^w$ 。

接下来, 我们看到函数 $U2T$ 描述了从无符号数到补码的转换, 而 $T2U$ 描述的是补码到无符号的转换。这两个函数描述了在大多数 C 语言实现中这两种数据类型之间的强制类型转换效果。

 **练习题 2.19** 利用你解答练习题 2.17 时填写的表格, 填写下列描述函数 $T2U_4$ 的表格。

x	$T2U_4(x)$
-8	
-3	
-2	
-1	
0	
5	

通过上述这些例子, 我们可以看到给定位模式的补码与无符号数之间的关系可以表示为函数 $T2U$ 的一个属性:

原理: 补码转换为无符号数

对满足 $TMin_w \leq x \leq TMax_w$ 的 x 有:

$$T2U_w(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \geq 0 \end{cases} \quad (2.5)$$

比如, 我们看到 $T2U_{16}(-12\ 345) = -12\ 345 + 2^{16} = 53\ 191$, 同时 $T2U_w(-1) = -1 + 2^w = UMax_w$ 。

该属性可以通过比较公式(2.1)和公式(2.3)推导出来。

推导: 补码转换为无符号数

比较等式(2.1)和等式(2.3), 我们可以发现对于位模式 $\vec{x}$, 如果我们计算 $B2U_w(\vec{x}) - B2T_w(\vec{x})$ 之差, 从 0 到 $w-2$ 的位的加权和将互相抵消掉, 剩下一个值: $B2U_w(\vec{x}) - B2T_w(\vec{x}) = x_{w-1}(2^{w-1} - (-2^{w-1})) = x_{w-1}2^w$ 。这就得到一个关系: $B2U_w(\vec{x}) = x_{w-1}2^w + B2T_w(\vec{x})$ 。我们因此就有

$$B2U_w(T2B_w(x)) = T2U_w(x) = x + x_{w-1}2^w \quad (2.6)$$

根据公式(2.5)的两种情况, 在 x 的补码表示中, 位 x_{w-1} 决定了 x 是否为负。 ■

比如说, 图 2-16 比较了当 $w=4$ 时函数 $B2U$ 和 $B2T$ 是如何将数值变成位模式的。对补码来说, 最高有效位是符号位, 我们用带向左箭头的条来表示。对于无符号数来说, 最高有效位是正权重, 我们用带向右的箭头的条来表示。从补码变为无符号数, 最高有效位

的权重从 -8 变为 $+8$ 。因此，补码表示的负数如果看成无符号数，值会增加 $2^4=16$ 。因而， -5 变成了 $+11$ ，而 -1 变成了 $+15$ 。

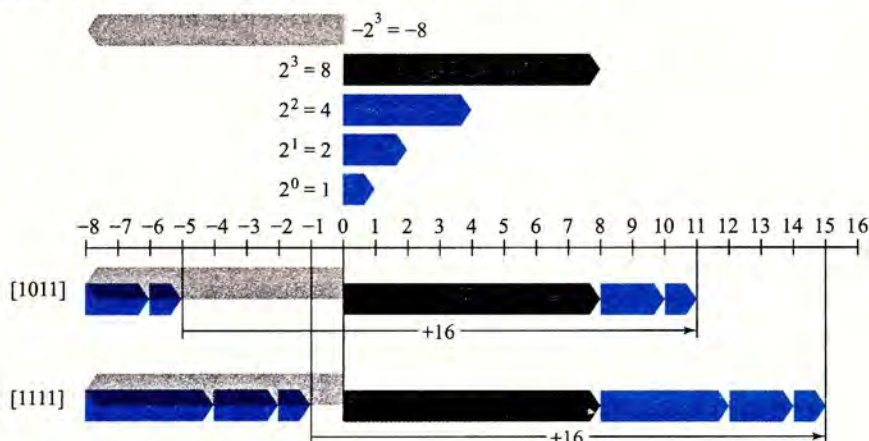


图 2-16 比较当 $w=4$ 时无符号表示和补码表示(对补码和无符号数来说，最高有效位的权重分别是 -8 和 $+8$ ，因而产生一个差为 16)

图 2-17 说明了函数 $T2U$ 的一般行为。如图所示，当将一个有符号数映射为它相应的无符号数时，负数就被转换成了大的正数，而非负数会保持不变。

练习题 2.20 请说明等式(2.5)是如何应用到解答练习题 2.19 时生成的表格中的各项的。

反过来看，我们希望推导出一个无符号数 u 和与之对应的有符号数 $U2T_w(u)$ 之间的关系：

原理：无符号数转换为补码

对满足 $0 \leq u \leq UMax_w$ 的 u 有：

$$U2T_w(u) = \begin{cases} u, & u \leq TMax_w \\ u - 2^w, & u > TMax_w \end{cases} \quad (2.7)$$

该原理证明如下：

推导：无符号数转换为补码

设 $\vec{u} = U2B_w(u)$ ，这个位向量也是 $U2T_w(u)$ 的补码表示。公式(2.1)和公式(2.3)结合起来有

$$U2T_w(u) = -u_{w-1}2^w + u \quad (2.8)$$

在 u 的无符号表示中，对公式(2.7)的两种情况来说，位 u_{w-1} 决定了 u 是否大于 $TMax_w = 2^{w-1} - 1$ 。 ■

图 2-18 说明了函数 $U2T$ 的行为。对于小的数($\leq TMax_w$)，从无符号到有符号的转换将保留数字的原值。对于大的数($> TMax_w$)，数字将被转换为一个负数值。

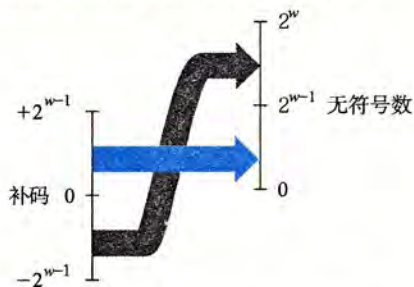


图 2-17 从补码到无符号数的转换。函数 $T2U$ 将负数转换为大的正数

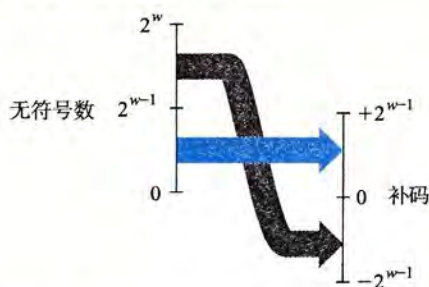


图 2-18 从无符号数到补码的转换。函数 $U2T$ 把大于 $2^{w-1}-1$ 的数字转换为负值

总结一下，我们考虑无符号与补码表示之间互相转换的结果。对于在范围 $0 \leq x \leq TMax_w$ 之内的值 x 而言，我们得到 $T2U_w(x) = x$ 和 $U2T_w(x) = x$ 。也就是说，在这个范围内的数字有相同的无符号和补码表示。对于这个范围以外的数值，转换需要加上或者减去 2^w 。例如，我们有 $T2U_w(-1) = -1 + 2^w = UMax_w$ ——最靠近 0 的负数映射为最大的无符号数。在另一个极端，我们可以看到 $T2U_w(TMin_w) = -2^{w-1} + 2^w = 2^{w-1} = TMax_w + 1$ ——最小的负数映射为一个刚好在补码的正数范围之外的无符号数。使用图 2-15 的示例，我们能看到 $T2U_{16}(-12\ 345) = 65\ 563 + (-12\ 345) = 53\ 191$ 。

2.2.5 C 语言中的有符号数与无符号数

如图 2-9 和图 2-10 所示，C 语言支持所有整型数据类型的有符号和无符号运算。尽管 C 语言标准没有指定有符号数要采用某种表示，但是几乎所有的机器都使用补码。通常，大多数数字都默认为是有符号的。例如，当声明一个像 12345 或者 0x1A2B 这样的常量时，这个值就被认为是有符号的。要创建一个无符号常量，必须加上后缀字符 ‘u’ 或者 ‘U’，例如，12345U 或者 0x1A2Bu。

C 语言允许无符号数和有符号数之间的转换。虽然 C 标准没有精确规定应如何进行这种转换，但大多数系统遵循的原则是底层的位表示保持不变。因此，在一台采用补码的机器上，当从无符号数转换为有符号数时，效果就是应用函数 $U2T_w$ ，而从有符号数转换为无符号数时，就是应用函数 $T2U_w$ ，其中 w 表示数据类型的位数。

显式的强制类型转换就会导致转换发生，就像下面的代码：

```
1      int tx, ty;
2      unsigned ux, uy;
3
4      tx = (int) ux;
5      uy = (unsigned) ty;
```

另外，当一种类型的表达式被赋值给另外一种类型的变量时，转换是隐式发生的，就像下面的代码：

```
1      int tx, ty;
2      unsigned ux, uy;
3
4      tx = ux; /* Cast to signed */
5      uy = ty; /* Cast to unsigned */
```

当用 printf 输出数值时，分别用指示符 %d、%u 和 %x 以有符号十进制、无符号十进制和十六进制格式输出一个数字。注意 printf 没有使用任何类型信息，所以它可以用指示符 %u 来输出类型为 int 的数值，也可以用指示符 %d 输出类型为 unsigned 的数值。例如，考虑下面的代码：

```
1      int x = -1;
2      unsigned u = 2147483648; /* 2 to the 31st */
3
4      printf("x = %u = %d\n", x, x);
5      printf("u = %u = %d\n", u, u);
```

当在一个 32 位机器上运行时，它的输出如下：

```
x = 4294967295 = -1
u = 2147483648 = -2147483648
```


在这两种情况下，printf 首先将这个字当作一个无符号数输出，然后把它当作一个有符号数输出。以下是实际运行中的转换函数： $T2U_{32}(-1) = UMax_{32} = 2^{32} - 1$ 和 $U2T_{32}(2^{31}) = 2^{31} - 2^{32} = -2^{31} = TMin_{32}$ 。

由于 C 语言对同时包含有符号和无符号数表达式的这种处理方式，出现了一些奇特的行为。当执行一个运算时，如果它的一个运算数是有符号的而另一个是无符号的，那么 C 语言会隐式地将有符号参数强制类型转换为无符号数，并假设这两个数都是非负的，来执行这个运算。就像我们将要看到的，这种方法对于标准的算术运算来说并无多大差异，但是对于像<和>这样的关系运算符来说，它会导致非直观的结果。图 2-19 展示了一些关系表达式的示例以及它们得到的求值结果，这里假设数据类型 int 表示为 32 位补码。考虑比较式 $-1 < 0U$ 。因为第二个运算数是无符号的，第一个运算数就会被隐式地转换为无符号数，因此表达式就等价于 $4294967295U < 0U$ (回想 $T2U_w(-1) = UMax_w$)，这个答案显然是错的。其他那些示例也可以通过相似的分析来理解。

表 达 式	类 型	求 值
0 == 0U	无符号	1
-1 < 0	有符号	1
-1 < 0U	无符号	0*
2147483647 > -2147483647-1	有符号	1
2147483647U > -2147483647-1	无符号	0*
2147483647 > (int) 2147483648U	有符号	1*
-1 > -2	有符号	1
(unsigned) -1 > -2	无符号	1

图 2-19 C 语言的升级规则的效果

注：非直观的情况标注了 ‘*’。当一个运算数是无符号的时候，另一个运算数也被隐式强制转换为无符号。
将 $TMin_{32}$ 写为 $-2147483647-1$ 的原因请参见网络旁注 DATA:TMIN。

 **练习题 2.21** 假设在采用补码运算的 32 位机器上对这些表达式求值，按照图 2-19 的格式填写下表，描述强制类型转换和关系运算的结果。

表 达 式	类 型	求 值
$-2147483647-1 == 2147483648U$		
$-2147483647-1 < 2147483647$		
$-2147483647-1U < 2147483647$		
$-2147483647-1 < -2147483647$		
$-2147483647-1U < -2147483647$		

网络旁注 DATA:TMIN C 语言中 TMin 的写法

在图 2-19 和练习题 2.21 中，我们很小心地将 $TMin_{32}$ 写成 $-2147483647-1$ 。为什么不简单地写成 -2147483648 或者 $0x80000000$ ？看一下 C 头文件 limits.h，注意到它们使用了跟我们写 $TMin_{32}$ 和 $TMax_{32}$ 类似的方法：

```
/* Minimum and maximum values a 'signed int' can hold. */
#define INT_MAX 2147483647
#define INT_MIN (~INT_MAX - 1)
```

不幸的是，补码表示的不对称性和 C 语言的转换规则之间奇怪的交互，迫使我们用

这种不寻常的方式来写 $TMin_{32}$ 。虽然理解这个问题需要我们钻研 C 语言标准的一些比较隐晦的角落，但是它能够帮助我们充分领会整数数据类型和表示的一些细微之处。

2.2.6 扩展一个数字的位表示

一个常见的运算是在不同字长的整数之间转换，同时又保持数值不变。当然，当目标数据类型太小以至于不能表示想要的值时，这根本就是不可能的。然而，从一个较小的数据类型转换到一个较大的类型，应该总是可能的。

要将一个无符号数转换为一个更大的数据类型，我们只要简单地在表示的开头添加 0。这种运算被称为零扩展(zero extension)，表示原理如下：

原理：无符号数的零扩展

定义宽度为 w 的位向量 $\vec{u} = [u_{w-1}, u_{w-2}, \dots, u_0]$ 和宽度为 w' 的位向量 $\vec{u}' = [0, \dots, 0, u_{w-1}, u_{w-2}, \dots, u_0]$ ，其中 $w' > w$ 。则 $B2U_w(\vec{u}) = B2U_{w'}(\vec{u}')$ 。

按照公式(2.1)，该原理可以看作是直接遵循了无符号数编码的定义。

要将一个补码数字转换为一个更大的数据类型，可以执行一个符号扩展(sign extension)，在表示中添加最高有效位的值，表示为如下原理。我们用蓝色标出符号位 x_{w-1} 来突出它在符号扩展中的角色。

原理：补码数的符号扩展

定义宽度为 w 的位向量 $\vec{x} = [x_{w-1}, x_{w-2}, \dots, x_0]$ 和宽度为 w' 的位向量 $\vec{x}' = [x_{w-1}, \dots, x_{w-1}, x_{w-1}, x_{w-2}, \dots, x_0]$ ，其中 $w' > w$ 。则 $B2T_w(\vec{x}) = B2T_{w'}(\vec{x}')$ 。

例如，考虑下面的代码：

```
1  short sx = -12345;          /* -12345 */
2  unsigned short usx = sx;    /* 53191 */
3  int x = sx;                 /* -12345 */
4  unsigned ux = usx;          /* 53191 */
5
6  printf("sx = %d:\t", sx);
7  show_bytes((byte_pointer) &sx, sizeof(short));
8  printf("usx = %u:\t", usx);
9  show_bytes((byte_pointer) &usx, sizeof(unsigned short));
10 printf("x = %d:\t", x);
11 show_bytes((byte_pointer) &x, sizeof(int));
12 printf("ux = %u:\t", ux);
13 show_bytes((byte_pointer) &ux, sizeof(unsigned));
```

在采用补码表示的 32 位大端法机器上运行这段代码时，打印出如下输出：

```
sx = -12345:  cf c7
usx = 53191:  cf c7
x = -12345:  ff ff cf c7
ux = 53191:   00 00 cf c7
```

我们看到，尽管 -12 345 的补码表示和 53 191 的无符号表示在 16 位字长时是相同的，但是在 32 位字长时却是不同的。特别地，-12 345 的十六进制表示为 0xFFFFCFC7，而 53 191 的十六进制表示为 0x0000CFC7。前者使用的是符号扩展——最开头加了 16 位，都是最高有效位 1，表示为十六进制就是 0xFFFF。后者开头使用 16 个 0 来扩展，表示为十六进制就是 0x0000。

图 2-20 给出了从字长 $w=3$ 到 $w=4$ 的符号扩展的结果。位向量[101]表示值 $-4+1=$

-3。对它应用符号扩展,得到位向量[1101],表示的值 $-8+4+1=-3$ 。我们可以看到,对于 $w=4$,最高两位的组合值是 $-8+4=-4$,与 $w=3$ 时符号位的值相同。类似地,位向量[111]和[1111]都表示值-1。

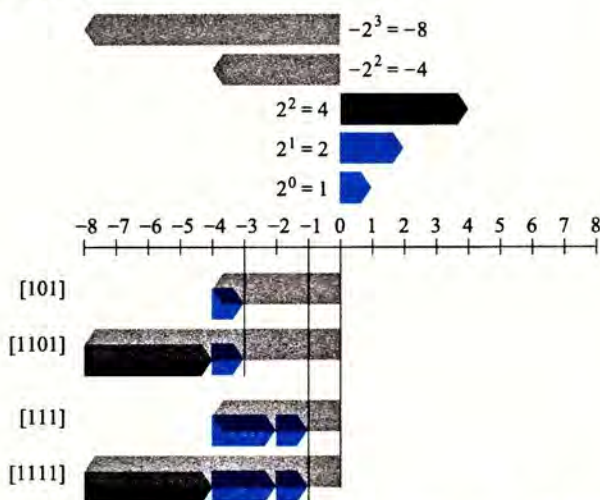


图 2-20 从 $w=3$ 到 $w=4$ 的符号扩展示例。对于 $w=4$, 最高两位组合权重为 $-8+4=-4$, 与 $w=3$ 时的符号位的权重一样

有了这个直觉,我们现在可以展示保持补码值的符号扩展。

推导: 补码数值的符号扩展

令 $w' = w + k$, 我们想要证明的是

$$B2T_{w+k}(\underbrace{[x_{w-1}, \dots, x_{w-1}]}_{k \text{ 次}}, x_{w-1}, x_{w-2}, \dots, x_0) = B2T_w([x_{w-1}, x_{w-2}, \dots, x_0])$$

下面的证明是对 k 进行归纳。也就是说,如果我们能够证明符号扩展一位保持了数值不变,那么符号扩展任意位都能保持这种属性。因此,证明的任务就变为了:

$$B2T_{w+1}([x_{w-1}, x_{w-1}, x_{w-2}, \dots, x_0]) = B2T_w([x_{w-1}, x_{w-2}, \dots, x_0])$$

用等式(2.3)展开左边的表达式,得到:

$$\begin{aligned} B2T_{w+1}([x_{w-1}, x_{w-1}, x_{w-2}, \dots, x_0]) &= -x_{w-1}2^w + \sum_{i=0}^{w-1} x_i 2^i \\ &= -x_{w-1}2^w + x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \\ &= -x_{w-1}(2^w - 2^{w-1}) + \sum_{i=0}^{w-2} x_i 2^i \\ &= -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i \\ &= B2T_w([x_{w-1}, x_{w-2}, \dots, x_0]) \end{aligned}$$

我们使用的关键属性是 $2^w - 2^{w-1} = 2^{w-1}$ 。因此,加上一个权值为 -2^w 的位,和将一个权值为 -2^{w-1} 的位转换为一个权值为 2^{w-1} 的位,这两项运算的综合效果就会保持原始的数值。 ■

 **练习题 2.22** 通过应用等式(2.3),表明下面每个位向量都是-5的补码表示。

A. [1011]

B. [11011]

C. [111011]

可以看到第二个和第三个位向量可以通过对第一个位向量做符号扩展得到。

值得一提的是，从一个数据大小到另一个数据大小的转换，以及无符号和有符号数字之间的转换的相对顺序能够影响一个程序的行为。考虑下面的代码：

```
1  short sx = -12345;          /* -12345  */
2  unsigned uy = sx;           /* Mystery! */
3
4  printf("uy = %u:\t", uy);
5  show_bytes((byte_pointer) &uy, sizeof(unsigned));
```

在一台大端法机器上，这部分代码产生如下输出：

```
uy = 4294954951:  ff ff cf c7
```

这表明当把 short 转换成 unsigned 时，我们先要改变大小，之后再完成从有符号到无符号的转换。也就是说 (unsigned) sx 等价于 (unsigned) (int) sx，求值得到 4 294 954 951，而不等价于 (unsigned) (unsigned short) sx，后者求值得到 53 191。事实上，这个规则是 C 语言标准要求的。

 练习题 2.23 考虑下面的 C 函数：

```
int fun1(unsigned word) {
    return (int) ((word << 24) >> 24);
}

int fun2(unsigned word) {
    return ((int) word << 24) >> 24;
}
```

假设在一个采用补码运算的机器上以 32 位程序来执行这些函数。还假设有符号数值的右移是算术右移，而无符号数值的右移是逻辑右移。

A. 填写下表，说明这些函数对几个示例参数的结果。你会发现用十六进制表示来会更方便，只要记住十六进制数字 8 到 F 的最高有效位等于 1。

w	fun1(w)	fun2(w)
0x00000076		
0x87654321		
0x000000C9		
0xEDCBA987		

B. 用语言来描述这些函数执行的有用的计算。

2.2.7 截断数字

假设我们不用额外的位来扩展一个数值，而是减少表示一个数字的位数。例如下面代码中这种情况：

```
1  int x = 53191;
2  short sx = (short) x; /* -12345 */
3  int y = sx;           /* -12345 */
```

当我们把 x 强制类型转换为 short 时，我们就将 32 位的 int 截断为了 16 位的 short int。

就像前面所看到的, 这个 16 位的位模式就是 -12 345 的补码表示。当我们把它强制类型转换回 int 时, 符号扩展把高 16 位设置为 1, 从而生成 -12 345 的 32 位补码表示。

当将一个 w 位的数 $\vec{x} = [x_{w-1}, x_{w-2}, \dots, x_0]$ 截断为一个 k 位数字时, 我们会丢弃高 $w-k$ 位, 得到一个位向量 $\vec{x}' = [x_{k-1}, x_{k-2}, \dots, x_0]$ 。截断一个数字可能会改变它的值——溢出的一种形式。对于一个无符号数, 我们可以很容易得出其数值结果。

原理: 截断无符号数

令 $\vec{x}$ 等于位向量 $[x_{w-1}, x_{w-2}, \dots, x_0]$, 而 $\vec{x}'$ 是将其截断为 k 位的结果: $\vec{x}' = [x_{k-1}, x_{k-2}, \dots, x_0]$ 。令 $x = B2U_w(\vec{x})$, $x' = B2U_k(\vec{x}')$ 。则 $x' = x \bmod 2^k$ 。

该原理背后的直觉就是所有被截去的位其权重形式都为 2^i , 其中 $i \geq k$, 因此, 每一个权在取模操作下结果都为零。可用如下推导表示:

推导: 截断无符号数

通过对等式(2.1)应用取模运算就可以看到:

$$\begin{aligned} B2U_w([x_{w-1}, x_{w-2}, \dots, x_0]) \bmod 2^k &= \left[\sum_{i=0}^{w-1} x_i 2^i \right] \bmod 2^k \\ &= \left[\sum_{i=0}^{k-1} x_i 2^i \right] \bmod 2^k \\ &= \sum_{i=0}^{k-1} x_i 2^i \\ &= B2U_k([x_{k-1}, x_{k-2}, \dots, x_0]) \end{aligned}$$

在这段推导中, 我们利用了属性: 对于任何 $i \geq k$, $2^i \bmod 2^k = 0$ 。 ■

补码截断也具有相似的属性, 只不过要将最高位转换为符号位:

原理: 截断补码数值

令 $\vec{x}$ 等于位向量 $[x_{w-1}, x_{w-2}, \dots, x_0]$, 而 $\vec{x}'$ 是将其截断为 k 位的结果: $\vec{x}' = [x_{k-1}, x_{k-2}, \dots, x_0]$ 。令 $x = B2U_w(\vec{x})$, $x' = B2T_k(\vec{x}')$ 。则 $x' = U2T_k(x \bmod 2^k)$ 。

在这个公式中, $x \bmod 2^k$ 将是 0 到 $2^k - 1$ 之间的一个数。对其应用函数 $U2T_k$ 产生的效果是把最高有效位 x_{k-1} 的权重从 2^{k-1} 转变为 -2^{k-1} 。举例来看, 将数值 $x = 53\,191$ 从 int 转换为 short。由于 $2^{16} = 65\,536 \geq x$, 我们有 $x \bmod 2^{16} = x$ 。但是, 当我们把这个数转换为 16 位的补码时, 我们得到 $x' = 53\,191 - 65\,536 = -12\,345$ 。

推导: 截断补码数值

使用与无符号数截断相同的参数, 则有

$$B2U_w([x_{w-1}, x_{w-2}, \dots, x_0]) \bmod 2^k = B2U_k([x_{k-1}, x_{k-2}, \dots, x_0])$$

也就是, $x \bmod 2^k$ 能够被一个位级表示为 $[x_{k-1}, x_{k-2}, \dots, x_0]$ 的无符号数表示。将其转换为补码数则有 $x' = U2T_k(x \bmod 2^k)$ 。 ■

总而言之, 无符号数的截断结果是:

$$B2U_k([x_{k-1}, x_{k-2}, \dots, x_0]) = B2U_w([x_{w-1}, x_{w-2}, \dots, x_0]) \bmod 2^k \quad (2.9)$$

而补码数字的截断结果是:

$$B2T_k([x_{k-1}, x_{k-2}, \dots, x_0]) = U2T_k(B2U_w([x_{w-1}, x_{w-2}, \dots, x_0]) \bmod 2^k) \quad (2.10)$$



练习题 2.24 假设将一个 4 位数值(用十六进制数字 0~F 表示)截断到一个 3 位数值(用十六进制数字 0~7 表示)。填写下表, 根据那些位模式的无符号和补码解释, 说明这种截断对某些情况的结果。

十六进制		无符号		补码	
原始值	截断值	原始值	截断值	原始值	截断值
0	0	0		0	
2	2	2		2	
9	1	9		-7	
B	3	11		-5	
F	7	15		-1	

解释如何将等式(2.9)和等式(2.10)应用到这些示例上。

2.2.8 关于有符号数与无符号数的建议

就像我们看到的那样，有符号数到无符号数的隐式强制类型转换导致了某些非直观的行为。而这些非直观的特性经常导致程序错误，并且这种包含隐式强制类型转换的细微差别的错误很难被发现。因为这种强制类型转换是在代码中没有明确指示的情况下发生的，程序员经常忽视了它的影响。

下面两个练习题说明了某些由于隐式强制类型转换和无符号数据类型造成的细微的错误。

 **练习题 2.25** 考虑下列代码，这段代码试图计算数组 a 中所有元素的和，其中元素的数量由参数 length 给出。

```
1  /* WARNING: This is buggy code */
2  float sum_elements(float a[], unsigned length) {
3      int i;
4      float result = 0;
5
6      for (i = 0; i <= length-1; i++)
7          result += a[i];
8      return result;
9  }
```

当参数 length 等于 0 时，运行这段代码应该返回 0.0。但实际上，运行时会遇到一个内存错误。请解释为什么会发生这样的情况，并且说明如何修改代码。

 **练习题 2.26** 现在给你一个任务，写一个函数用来判定一个字符串是否比另一个更长。前提是你要用字符串库函数 strlen，它的声明如下：

```
/* Prototype for library function strlen */
size_t strlen(const char *s);
```

最开始你写的函数是这样的：

```
/* Determine whether string s is longer than string t */
/* WARNING: This function is buggy */
int strlonger(char *s, char *t) {
    return strlen(s) - strlen(t) > 0;
}
```

当你在一些示例数据上测试这个函数时，一切似乎都是正确的。进一步研究发现在头文件 stdio.h 中数据类型 size_t 是定义成 unsigned int 的。

- A. 在什么情况下，这个函数会产生不正确的结果？
- B. 解释为什么会出现这样不正确的结果。
- C. 说明如何修改这段代码好让它能可靠地工作。

旁注 函数 `getpeername` 的安全漏洞

2002年,从事 FreeBSD 开源操作系统项目的程序员意识到,他们对 `getpeername` 函数的实现存在安全漏洞。代码的简化版本如下:

```

1  /*
2   * Illustration of code vulnerability similar to that found in
3   * FreeBSD's implementation of getpeername()
4   */
5
6  /* Declaration of library function memcpy */
7  void *memcpy(void *dest, void *src, size_t n);
8
9  /* Kernel memory region holding user-accessible data */
10 #define KSIZE 1024
11 char kbuf[KSIZE];
12
13 /* Copy at most maxlen bytes from kernel region to user buffer */
14 int copy_from_kernel(void *user_dest, int maxlen) {
15     /* Byte count len is minimum of buffer size and maxlen */
16     int len = KSIZE < maxlen ? KSIZE : maxlen;
17     memcpy(user_dest, kbuf, len);
18     return len;
19 }
```

在这段代码里,第7行给出的是库函数 `memcpy` 的原型,这个函数是要将一段指定长度为 `n` 的字节从内存的一个区域复制到另一个区域。

从第14行开始的函数 `copy_from_kernel` 是要将一些操作系统内核维护的数据复制到指定的用户可以访问的内存区域。对用户来说,大多数内核维护的数据结构应该是不可读的,因为这些数据结构可能包含其他用户和系统上运行的其他作业的敏感信息,但是显示为 `kbuf` 的区域是用户可以读的。参数 `maxlen` 给出的是分配给用户的缓冲区的长度,这个缓冲区是用参数 `user_dest` 指示的。然后,第16行的计算确保复制的字节数据不会超出源或者目标缓冲区可用的范围。

不过,假设有些怀有恶意的程序员在调用 `copy_from_kernel` 的代码中对 `maxlen` 使用了负数值,那么,第16行的最小值计算会把这个值赋给 `len`,然后 `len` 会作为参数 `n` 被传递给 `memcpy`。不过,请注意参数 `n` 是被声明为数据类型 `size_t` 的。这个数据类型是在库文件 `stdio.h` 中(通过 `typedef`)被声明的。典型地,对32位程序它被定义为 `unsigned int`,对64位程序定义为 `unsigned long`。既然参数 `n` 是无符号的,那么 `memcpy` 会把它当作一个非常大的正整数,并且试图将这样多字节的数据从内核区域复制到用户的缓冲区。虽然复制这么多字节(至少 2^{31} 个)实际上不会完成,因为程序会遇到进程中非法地址的错误,但是程序还是能读到它没有被授权的内核内存区域。

我们可以看到,这个问题是由于数据类型的不匹配造成的:在一个地方,长度参数是有符号数;而另一个地方,它又是无符号数。正如这个例子表明的那样,这样的不匹配会成为缺陷的原因,甚至会导致安全漏洞。幸运的是,还没有案例报告有程序员在 FreeBSD 上利用了这个漏洞。他们发布了一个安全建议,“FreeBSD-SA-02:38.signed-error”,建议系统管理员如何应用补丁消除这个漏洞。要修正这个缺陷,只要将 `copy_from_kernel` 的参数 `maxlen` 声明为类型 `size_t`,也就是与 `memcpy` 的参数 `n` 一致。同时,我们也应该将本地变量 `len` 和返回值声明为 `size_t`。

我们已经看到了许多无符号运算的细微特性，尤其是有符号数到无符号数的隐式转换，会导致错误或者漏洞的方式。避免这类错误的一种方法就是绝不使用无符号数。实际上，除了 C 以外很少有语言支持无符号整数。很明显，这些语言的设计者认为它们带来的麻烦要比益处多得多。比如，Java 只支持有符号整数，并且要求以补码运算来实现。正常的右移运算符 $\gg$ 被定义为执行算术右移。特殊的运算符 $\ggg$ 被指定为执行逻辑右移。

当我们想要把字仅仅看做是位的集合而没有任何数字意义时，无符号数值是非常有用的。例如，往一个字中放入描述各种布尔条件的标记(flag)时，就是这样。地址自然地就是无符号的，所以系统程序员发现无符号类型是很有帮助的。当实现模运算和多精度运算的数学包时，数字是由字的数组来表示的，无符号值也会非常有用。

2.3 整数运算

许多刚入门的程序员非常惊奇地发现，两个正数相加会得出一个负数，而比较表达式 $x < y$ 和比较表达式 $x - y < 0$ 会产生不同的结果。这些属性是由于计算机运算的有限性造成的。理解计算机运算的细微之处能够帮助程序员编写更可靠的代码。

2.3.1 无符号加法

考虑两个非负整数 x 和 y ，满足 $0 \leq x, y < 2^w$ 。每个数都能表示为 w 位无符号数字。然而，如果计算它们的和，我们就有一个可能的范围 $0 \leq x + y \leq 2^{w+1} - 2$ 。表示这个和可能需要 $w+1$ 位。例如，图 2-21 展示了当 x 和 y 有 4 位表示时，函数 $x+y$ 的坐标图。参数(显示在水平轴上)取值范围为 0~15，但是和的取值范围为 0~30。函数的形状是一个有坡度的平面(在两个维度上，函数都是线性的)。如果保持和为一个 $w+1$ 位的数字，并且把它加上另外一个数值，我们可能需要 $w+2$ 个位，以此类推。这种持续的“字长膨胀”意味着，要想完整地表示算术运算的结果，我们不能对字长做任何限制。一些编程语言，例如 Lisp，实际上就支持无限精度的运算，允许任意的(当然，要在机器的内存限制之内)整数运算。更常见的是，编程语言支持固定精度的运算，因此像“加法”和“乘法”这样的运算不同于它们在整数上的相应运算。

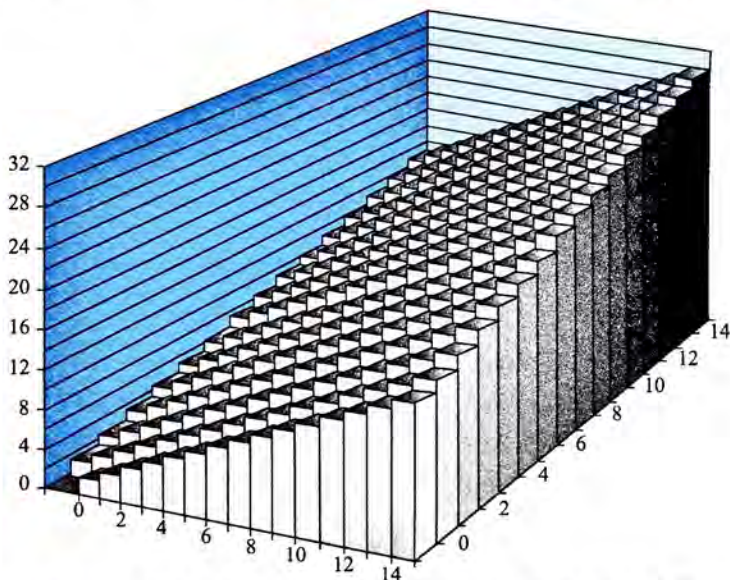


图 2-21 整数加法。对于一个 4 位的字长，其和可能需要 5 位